Self-Improving Agent System — Consolidated Best Plan

Scope: One closed-loop agent improvement system (Autonomous Improvement Kernel) shared by HunterXJob (JobTomatik), Allo, Fidelis, Pedit, and Agent Gang.

---

1. Verdict

After deduplicating the research (7 overlapping answers, ~2 unique responses plus progressively more concrete refinements), **all variants converge on the same core plan**, and the final, most refined version is the correct one:

> **Build one shared Autonomous Improvement Kernel. Prove the full learning loop offline on Fidelis first, then connect Pedit, Allo, Agent Gang, and HunterXJob through per-project adapters — with champion/challenger promotion, gated code authoring, and human approval for risky actions.**

The research's own final decision section is canonical; earlier answers are supporting detail (SQL schemas, judge prompts, taxonomy) that slot into it. What the research calls "self-improvement" is **harness-level evolution** — versioned prompts, tool schemas, routing, retry policies, configs, and later code patches — never live self-rewriting in the request path.

---

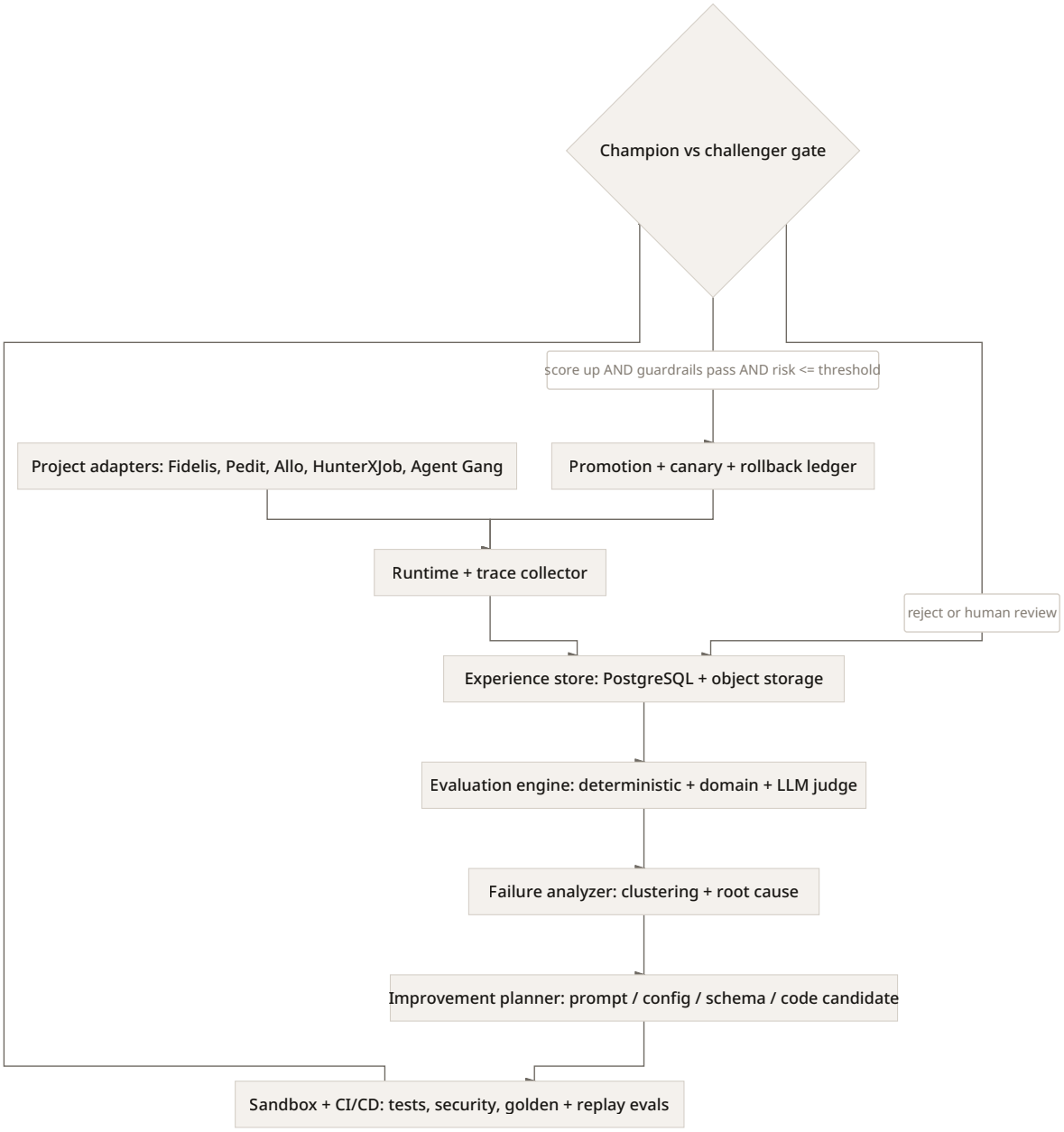
2. What every plan in the research agrees on (consensus kernel)

These points appear in every variant and should be treated as settled:

1. **The loop:** run → trace → grade → diagnose → propose candidate → sandbox test → champion/challenger compare → promote or reject → repeat.
2. **Traces over final answers:** capture model calls, tool calls, arguments, results, errors, retries, latency, cost — because failure is usually in an intermediate step, not the final output.
3. **Three grader layers:** deterministic checks first, domain metrics second, one calibrated LLM-as-judge (structured JSON, fixed rubric) last. The judge is never the sole promotion authority.
4. **Champion/challenger:** the live app runs only the champion; a separate offline worker creates challengers. Ties keep the champion.
5. **Two datasets:** a frozen golden set (never auto-modified) + a rolling replay set of recent real failures.
6. **Gated code authoring:** the system may *write* code continuously, but CI/CD — not the model — decides deployment. Sandbox has no production credentials/secrets.
7. **Risk policy per project:** prompts/schemas/routing auto-change; auth, payments, live submissions, phone calls, destructive actions stay human-gated.
8. **Optimizer stability:** the improvement worker's own instructions stay static; it changes project harnesses, never itself.
9. **Defects become regression tests:** every confirmed failure cluster becomes a test so the same defect is never rediscovered.
10. **Fine-tuning comes later,** only after a clean reviewed dataset of successes/failures exists. Harness-level learning pays off much earlier.

---

3. Consolidated architecture



Promotion rule (enforced in code, not by model judgment):

```
Promote(c) ⟺ Score(c) > Score(champion)
 ∧ TestsPass(c)
 ∧ GuardrailsPass(c)
 ∧ RegressionCount = 0
 ∧ Latency ≤ budget
 ∧ Cost ≤ budget
 ∧ Risk ≤ threshold
```

If the candidate ties the champion, keep the champion. If the judge disagrees with a deterministic safety check, reject or escalate to human review.

4. Rollout order (risk-ranked)

Order	Project	Why	Mode
1	**Fidelis**	Offline evaluation, reproducible inputs, strong quality metrics, zero external side effects	Full loop first
2	**Pedit**	Offline with strict file-integrity/layout checks	Full loop
3	**Allo**	Measurable real-time metrics, but privacy/consent/ telephony need stronger controls	Loop on prompts + audio pipeline only
4	**Agent Gang**	Multi-agent handoffs add failure modes; start with one orchestrator	Loop on tool choice + delegation
5	**HunterXJob**	Highest external-action risk (submits applications, touches accounts)	Dry-run + review package; submission stays human-gated

Per-project improvement scopes (from the research, kept as-is):

- **Fidelis** — auto-improve: model routing, restoration profile, tile size/overlap, artifact thresholds, confidence gating. Penalize hallucinated texture, ringing, plastic skin; never optimize for sharpness alone. Success = source fidelity, not invented detail.
- **Pedit** — auto-improve: operation planning, bounding-box detection, font fitting, layout heuristics, parser adapters. A candidate that corrupts the PDF is rejected even if the edit itself was correct.
- **Allo** — auto-improve: prompt style, response length, buffering, barge-in thresholds, session recovery, memory retrieval. Consent, recording, identity, privacy, emergency behavior stay outside self-modification.
- **Agent Gang** — start with one orchestrator; measure tool selection, argument validity, unnecessary delegation, handoff failures, cost. Add agents only when eval data proves specialization wins.
- **HunterXJob** — auto-improve: selectors, extraction, field mapping, answer validation, duplicate detection, retry logic, ranking. Human approval for: final submission, legal agreements, sensitive demographic answers, employer messages, credentials. A successful button click is not proof an application was received — require independent confirmation.

5. My adjustments to the research plan

The research is sound; these four refinements make it cheaper, safer, and faster to first value:

5.1 Decouple tracing from promotion

The research implies building the kernel against Fidelis only at first. Instead:

- **Instrument all five projects with tracing from day one.** Tracing is read-only and zero-risk, and old traces are what the failure analyzer needs later. Retro-fitting tracing is the most common reason these systems stall.
- **Roll out *promotion capability* in the risk order above.** Fidelis gets the full closed loop first; the other projects merely emit traces until their adapter is ready.

5.2 Harness = versioned config before code

The evolving "harness" should start as **declarative versioned config** (prompts, tool descriptions, JSON schemas, routing weights, retry policies, thresholds in YAML/JSON under git), not code. That makes every early candidate a config diff — trivially reviewable, revertible, and safe. Only after the config-level loop has earned trust (several clean promotions) do you enable the code-writing agent for parser adapters, layout

heuristics, selector repair, and tests. The research agrees in principle; this makes it an explicit gate with a measurable bar.

5.3 Kernel v0 is one package, not a platform

The research's repo layouts are right structurally, but v0 should be:

- **One Python package** (`improvement_kernel`) importable by each app + **one Postgres database** + **one background worker** + **GitHub Actions** for the sandbox/promotion gate.
- No event bus, no message queue, no separate API/dashboard services until volume demands them. The research's `apps/api`, `workers/*_worker.py`, and dashboard are v1 concerns. This matches the pragmatic, fail-closed, cost-effective constraint set.
- Postgres schema (from the research): `agent_versions`, `runs`, `traces`, `grades`, `defects`, `evaluations`, `promotions`. Keep artifacts (images, PDFs, screenshots, patches) in object storage with hashes in Postgres.

5.4 Human-gate defaults, with an owner-only autonomy flag

For HunterXJob (JobTomatik), keep the default policy that live submissions require human approval — and keep the fully autonomous mode restricted to the owner's private runs, never offered publicly. This is both the research's risk recommendation and a standing project constraint. Practically: the policy file gets an `autonomy: owner_only_full` flag that is never enabled for any public/subscription tier.

---

6. Build sequence (consolidated, ~30 days)

Phase	Build	Success condition
Days 1–7: Tracing	Runtime wrapper, run/trace IDs, redaction, Postgres schema, artifact storage, trace viewer; wire tracing into all 5 apps (read-only)	One Fidelis task produces one complete, inspectable trace
Days 8–14: Evaluation	Fidelis adapter, deterministic graders, one fixed LLM judge, failure taxonomy, golden dataset (20–100 cases) + replay set	System reliably separates acceptable vs unacceptable output; judge calibrated against ~30 human-labeled examples
Days 15–21: Improvement	Failure clustering, defect reports, candidate generation (config-level), champion/challenger comparison, defect→regression-test conversion	System proposes a change and measures whether it helped
Days 22–30: CI/CD	Sandbox candidate branches, patch application, tests + security/secret scans, protected PR flow, staging + rollback ledger	System generates and tests a patch without any production access
After:	Pedit adapter → Allo adapter → Agent Gang adapter → HunterXJob adapter (dry-run first); enable code-writing agent only after repeated clean config-level promotions	Each project passes its first full loop

First end-to-end milestone (prove this before anything else):

```
one Fidelis image
→ restore with champion profile
→ complete trace
→ deterministic grader + judge
→ one defect recorded
→ one config candidate proposed
```

```
→ sandbox evaluation vs golden set
→ champion/challenger decision
→ one rollback-capable promotion
```

---

7. Auto-change vs approval boundary (all projects)

Auto-change allowed	Approval required
Prompts, tool descriptions, routing rules, retry strategy, ranking weights, JSON schemas, validation thresholds, test generation, failure clustering, non-destructive config	Auth, permissions, secrets, payments, data deletion, DB migrations, live job submissions, phone calls, recording/ consent, destructive overwrites, new external side effects, new write-capable tools

---

8. Key reference documentation (from the research)

- OpenAI Agent Evals guide
- OpenAI Trace Grading guide
- OpenAI Agent Improvement Loop cookbook
- Self-Evolving Agents: Autonomous Agent Retraining cookbook
- Evaluation Best Practices
- Agentic Governance cookbook
- A Practical Guide to Building Agents (PDF)
- OTel GenAI agent semantic conventions
- GitHub Actions deployment environments (approval gating)

---

9. The governing principle

The system should continuously **learn**, **propose**, and **test** — and only continuously **deploy** when a candidate has measurable evidence it is safer and better than the champion. The biggest mistake in the research is trying to build "an AI that improves itself" before building **measurement**; without evals, self-improvement is self-randomization.

```
experience → trace → grade → defect → regression test
→ proposed fix → sandbox → evaluation → champion comparison
→ promotion or rejection → new experience
```

That is constant learning from every attempt, while staying testable, recoverable, and safe.